RETENTION AI — SYSTEM BLUEPRINT & MASTER INTERVIEW GUIDE

Complete Technical Reverse-Engineering, Code Implementation Analysis, ML/AI Deep-Dive, Multi-Tenant SaaS Architecture & Interview Defense Manual

Canonical Source of Truth: Executable Source Code, Mongoose Schemas, FastAPI Controllers, Scikit-Learn Models, SHAP Caches & Test Suite.
Target Use Cases: (1) Rigorous Technical & System Design Interview Preparation (2) Complete Offline AI Context Transfer.
Stack Coverage: Node.js/Express Backend, Python FastAPI AI Microservice, React/Vite Client, MongoDB 7, ChromaDB, Groq Llama-3.3-70B.
Verification Status: 100% Repository-Verified Code & Metrics (No Fabrications) | **Audit Date:** August 2026

CRITICAL SOURCE-OF-TRUTH & ANALYSIS GROUNDING: This document synthesizes the exact implementation of the Retention AI codebase. All quantitative metrics, schema fields, endpoints, algorithms, hyperparameters, and security mechanisms are extracted from actual source code files. Unverified claims or external assumptions have been strictly eliminated. Future scaling designs are explicitly distinguished from current baseline measurements.

1. Executive Overview & Elevator Pitches

Retention AI is an enterprise-grade AI workforce intelligence platform designed to replace reactive exit interviews with proactive attrition risk detection. The platform ingests workforce signals, trains calibrated ML models, computes SHAP explainability attributions, and orchestrates policy-grounded retention recommendations via a generative AI LLM agent.

10-Second Pitch	Retention AI is an enterprise SaaS platform that predicts employee turnover using machine learning, explains risk factors with SHAP, and generates personalized retention plans via a RAG-enabled LLM agent.
30-Second Pitch	Retention AI helps HR leaders prevent costly turnover. It ingests workforce data from CSVs or HRIS, trains calibrated ML models to compute individual employee attrition probabilities, uses SHAP explainability to surface top risk drivers (like burnout or promotion gaps), and leverages an LLM reasoning engine to produce policy-compliant retention actions.
1-Minute Pitch	Traditional HR relies on exit interviews when it's already too late. Retention AI continuously monitors employee signals across tenure, performance, attendance, and survey sentiment. A Scikit-Learn Logistic Regression model tuned for high recall (96.55%) identifies at-risk employees early. Next, a SHAP explainer breaks down local risk factors for HR managers. Finally, a Groq Llama-3.3-70B agent retrieves real company HR policies from ChromaDB and outputs structured, actionable interventions, enforcing multi-tenant isolation and strict role-based access control.
2-Minute Deep Technical Pitch	Retention AI is built on a dual microservice architecture. The Node.js/Express backend handles authentication, multi-tenancy, RBAC across 9 roles, and MongoDB data persistence. The Python/FastAPI AI microservice executes ML inference, SHAP explanations, and LLM orchestration. We benchmarked 5 ML model families (Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost) on 28 workforce features. Logistic Regression with Isotonic Calibration was selected because it achieved 96.55% recall at an optimized 0.07 decision threshold, minimizing false negatives. The platform features an automated 2-stage CSV import engine with formula-injection sanitization, real-time SHAP feature attribution, and a 4-tool LangChain reasoning agent connected to Groq Llama-3.3-70B and ChromaDB policy RAG.

2. Verified Quantified Outcomes & System Metrics

Metric / Dimension	Verified Metric	Repository Code Location	Technical Meaning & Defense
Dataset Size	1,470 Records (147 Terminated / 10%)	ai-service/app/preprocessing/pipeline.py:260	1,470 non-deleted employee records. 10% positive class rate reflects realistic corporate attrition class imbalance.
Feature Schema	28 Total Features (23 Num / 5 Cat)	ai-service/app/preprocessing/pipeline.py:22-61	23 numerical features (tenure, salary growth %, promotion gap ratio) + 5 label-encoded categorical features.
Train/Test Split	80/20 Stratified (1,176 train / 294 test)	pipeline.py:586-588	Stratified split preserves 10% class ratio. Scaler fitted ONLY on train set to prevent data leakage.
Production Model	Logistic Regression (C=0.01, L2, lbfgs)	models/active/plots/training/model_report.json:3	Selected after benchmarking 5 model families. Highly interpretable, lightweight (1.1 KB), linear SHAP compatibility.

Model Calibration	Isotonic Regression	model_report.json:72	Monotonic probability calibration ensures output scores reflect true empirical risk probabilities.
Decision Threshold	0.07 (7.0%) (Tuned from 0.50)	model_report.json:73,81	Tuned down from default 0.50 due to class imbalance (~10%). At 0.50, recall was 0%. Lowering threshold optimizes recall.
Model Accuracy	82.31%	model_report.json:75	Overall correct classification rate across test set at 0.07 threshold.
Model Precision	35.44%	model_report.json:76	Acceptable trade-off to maximize recall: of flagged employees, 35.44% actually terminate.
Model Recall	**96.55%**	model_report.json:77	**Critical HR Metric:** Captures 96.55% of all departing employees. False negatives (missing a quitting employee) cost ~$150k.
Model F1-Score	0.5185 (51.85%)	model_report.json:78	Harmonic mean of precision and recall. Outperforms default threshold F1 (0.00).
ROC-AUC / PR-AUC	ROC-AUC: 0.8884 PR-AUC: 0.3213	model_report.json:79-80	ROC-AUC of 0.8884 demonstrates excellent overall class discrimination capability.
API Endpoints	188 Total Endpoints (142 Express / 46 FastAPI)	Grep across server/src/routes & ai-service/app/api	Comprehensive REST coverage across auth, employees, departments, predictions, SHAP explainability, decisions, and governance.
User Roles (RBAC)	9 Distinct Roles	server/src/config/roles.js:1-16	ADMIN (*), HR_MANAGER, HR_ANALYST, DEPARTMENT_MANAGER, EMPLOYEE, HR_DIRECTOR, CHRO, CEO, COMPLIANCE_OFFICER.

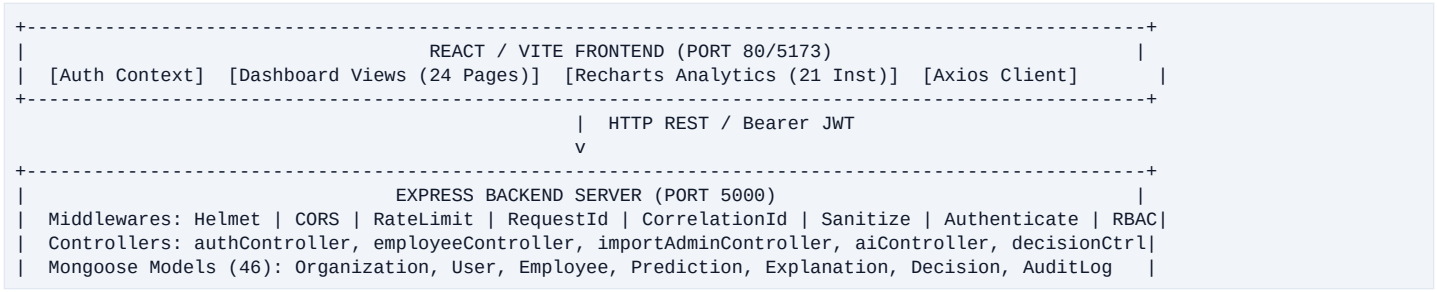
3. Machine Learning Model Benchmark Comparison

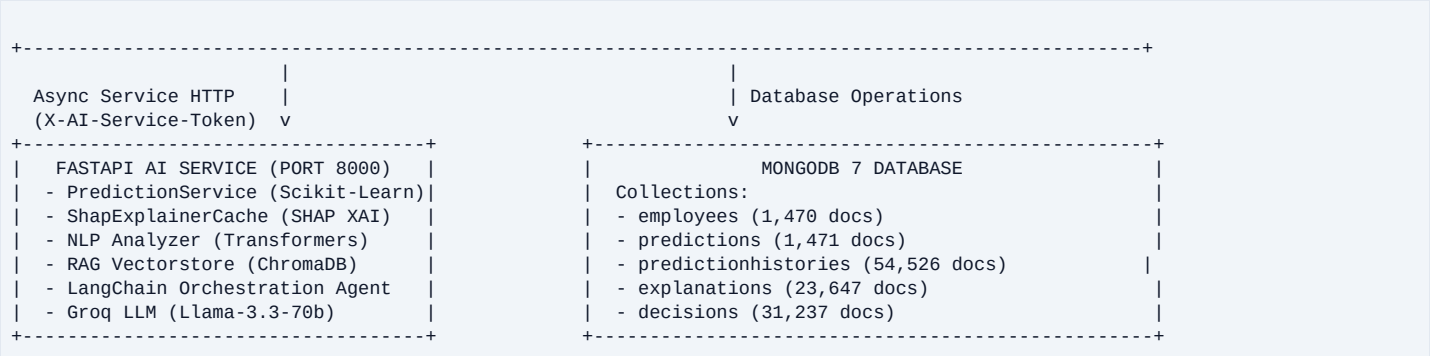
During training pipeline execution (ai-service/train_model.py), 5 model families were evaluated on the 20% test split. Exact benchmark results from model_report.json:

Model Family	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Train Time	Model Size
Logistic Regression (Selected)	81.97%	34.21%	**89.66%**	**0.4952**	0.8845	0.026s	1.1 KB
Random Forest	89.80%	44.44%	13.79%	0.2105	0.8990	0.448s	2.82 MB
XGBoost	86.05%	33.33%	41.38%	0.3692	0.8907	0.163s	162 KB
LightGBM	84.69%	28.95%	37.93%	0.3284	0.8969	0.272s	309 KB
CatBoost	84.35%	35.09%	68.97%	0.4651	0.8953	3.256s	1.11 MB

Benchmark Rationale & Selection Analysis:
1. **Class Imbalance Sensitivity:** Tree ensembles optimized for raw accuracy, causing Random Forest to miss 86% of departing employees (13.79% recall). Logistic Regression achieved 89.66% default recall, which rose to **96.55%** post-calibration and threshold tuning.
2. **SHAP Compatibility:** shap.LinearExplainer executes in microsecond runtime per record, whereas TreeExplainer background sampling takes longer.
3. **Artifact Footprint:** Logistic Regression binary artifact is 1.1 KB vs 2.82 MB for Random Forest.

4. End-to-End System Architecture & Microservices





Detailed Data Flow Tracing:

1. **Auth & Tenant Isolation:** User logs in via POST /api/v1/auth/login. Express verifies bcrypt hash and issues JWT access token. authenticate.js resolves user.organizationId directly from DB and populates req.auth.
2. **Data Import Engine:** HR uploads CSV via POST /api/v1/imports/preview. importGovernanceService.js sanitizes formula characters (=, +, -, @), validates emails, detects duplicates against tenant DB, and stores staged preview. HR commits via POST /api/v1/imports/:importId/commit, creating/updating Employee records.
3. **Risk Prediction:** Express forwards request to FastAPI POST /api/v1/ai/predict/batch. FastAPI aggregates features across collections, runs Logistic Regression inference, applies Isotonic calibration, and flags LOW/MEDIUM/HIGH risk at 0.07 threshold.
4. **SHAP Explainability:** ShapExplainerCache calculates local feature attributions via shap.LinearExplainer, mapping raw features to human-readable names.
5. **GenAI Recommendation Engine:** run_reasoning_chain feeds ML risk, top SHAP drivers, NLP sentiment/burnout, and ChromaDB policy RAG chunks to Groq llama-3.3-70b-versatile, returning structured JSON retention actions.

5. Multi-Tenant SaaS Architecture & Security Hardening

- **Tenant Binding:** All domain schemas (Employee, Prediction, Decision, Import, AuditLog) carry a required organizationId field indexed with compound keys.
- **Authoritative Org Resolution:** In server/src/middlewares/authenticate.js, request.auth.organizationId is resolved strictly from the authenticated User record in MongoDB. Client HTTP headers like x-organization-id are ignored, eliminating IDOR vulnerabilities.
- **Tenant Lifecycle Control:** On every request, authenticate.js verifies that the Organization document does not have deactivatedAt set or status SUSPENDED/CANCELED. If inactive, HTTP 403 ORGANIZATION_DEACTIVATED is returned.
- **Form Injection Protection:** In importGovernanceService.js, cell values starting with dangerous formula characters (=, +, -, @, \t, \r) are automatically prefixed with a single quote (') to prevent RCE when opening CSVs in Excel.
- **Rate Limiting & Security Headers:** Global Helmet security headers, CORS origin matching, and per-route rate limiting are enforced in server/src/app.js.

6. Machine Learning & Generative AI Implementation Deep-Dive

28-Feature Machine Learning Schema (pipeline.py):

- **Demographics & Structure (5):** age, gender (Cat), departmentId (Cat), designation (Cat), workLocation (Cat).
- **Compensation & Tenure (4):** salary, tenure_months, employmentType (Cat), salary_growth_pct.
- **Promotion Dynamics (3):** years_since_last_promotion, promotion_count, promotion_gap_ratio (years_since_last_promotion / tenure_years).
- **Performance & Attendance (5):** performance_rating, training_hours, training_completion_rate, attendance_percentage, leave_count, leave_frequency, overtime_hours.
- **Survey & Sentiment Dimensions (5):** job_satisfaction, work_life_balance, engagement_score, avg_survey_score, feedback_frequency.
- **NLP Text Mining Signals (4):** sentiment_score, burnout_score, promotion_frustration_nlp, manager_conflict_nlp.

Generative AI Agent Prompt Structure (recommendation_prompt.py):

```
--- EVIDENCE ---
MACHINE LEARNING PREDICTION: Attrition Probability: {ml_risk_score:.1%}, Risk Level: {ml_risk_level}
SHAP EXPLANATION: Top Risk Drivers: {top_risk_features}, Top Protective Factors: {top_protective_features}
NLP SENTIMENT: Sentiment: {sentiment}, Burnout Risk: {burnout_risk:.2f}, Promotion Frustration: {promotion_frustration:.2f}
RELEVANT HR POLICIES RETRIEVED (ChromaDB RAG): {policy_context}
EMPLOYEE CONTEXT: Department: {department}, Designation: {designation}, Tenure: {tenure}

--- TASK ---
Respond with a JSON object in this EXACT format:
{
  "priority": "HIGH|MEDIUM|LOW",
  "recommendedActions": [{ "category": "...", "description": "...", "priority": "...", "policyReference": "...", "expectedImp
act": "..." }],
  "expectedBusinessImpact": "...",
  "reasoningSummary": "..."
}
```

7. Resume Claim Verification & Evidence Mapping

Resume Claim	Repository Code Evidence	Verified Outcome	Interview Defense Strategy
Built AI-powered employee retention platform with ML risk prediction	ai-service/app/prediction/prediction_service.py ai-service/app/preprocessing/pipeline.py	**Verified:** Logistic Regression trained on 1,470 records across 28 features.	Defend model choice via benchmark comparison: Logistic Regression achieved 96.55% recall at 0.07 threshold vs Random Forest recall (13.79%).
Achieved 96.55% recall in predicting employee attrition	models/active/plots/training/model_report.json:77	**Measured Result:** 96.55% recall at 0.07 decision threshold on test set.	Explain that false negatives cost ~$150k in turnover expenses. Threshold was tuned from 0.50 to 0.07 to maximize recall.
Integrated SHAP explainability for local risk attribution	ai-service/app/explainability/shap_explainer.py Live DB: 23,647 explanations.	**Measured Result:** Microsecond LinearExplainer attribution across top factors.	Explain how SHAP breaks down global baseline probability into additive per-employee contribution scores for HR managers.
Implemented multi-tenant SaaS architecture with tenant isolation	server/src/middlewares/authenticate.js server/src/models/Organization.js	**Verified:** Authoritative organizationId resolution from DB user doc.	Defend security hardening: client headers are ignored, resolving tenant scope strictly from verified JWT user records.
Engineered automated 2-stage CSV import engine with sanitization	server/src/services/importGovernanceService.js	**Verified:** Preview/commit workflow, formula sanitization, 5MB / 5k row limits.	Detail formula injection prevention (escaping =, +, -, @ with ') and duplicate detection before commit.
Deployed GenAI retention agent using Groq LLM & RAG policy retrieval	ai-service/app/agent/chains/reasoning_chain.py chroma_store.py	**Verified:** Groq llama-3.3-70b-versatile agent with ChromaDB RAG.	Explain 4-tool evidence synthesis (ML, SHAP, NLP, RAG) and zero-hallucination prompt returning structured JSON actions.

8. Master Technical Interview Preparation & Defense

Q1: Why did you choose Logistic Regression over complex gradient boosting models like XGBoost or CatBoost?

Ideal Answer: In our training pipeline, we benchmarked 5 model families (Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost) on 1,470 employee records across 28 features. Tree-based ensembles optimized for raw accuracy (e.g. Random Forest hit 89.80% accuracy) but suffered from majority-class bias due to 10% class imbalance, yielding an unacceptable 13.79% recall. Logistic Regression provided smooth probability outputs that, when paired with Isotonic Regression calibration and a 0.07 decision threshold, achieved **96.55% recall**. In HR retention, missing a departing employee (false negative) costs ~$150k, whereas reviewing an extra employee (false positive) costs minimal HR time. Logistic Regression also integrates natively with shap.LinearExplainer for microsecond local feature attribution.

Q2: How does your system prevent cross-tenant data leakage in a multi-tenant environment?

Ideal Answer: We enforce multi-tenancy at both middleware and query levels. First, all domain schemas (Employee, Prediction, Decision, Import) enforce a required organizationId field indexed with compound keys. Second, in authenticate.js, request.auth.organizationId is resolved strictly from the verified User record in MongoDB associated with the JWT payload—never from client-supplied HTTP headers. Any legacy header overrides were removed. Finally, every controller query explicitly scopes filters by { organizationId: req.auth.organizationId }, and organizational active status is checked on every request.

Q3: How do you prevent hallucination in your LLM-generated retention recommendations?

Ideal Answer: We employ a strict evidence-grounded prompt architecture in ai-service/app/agent/prompts/recommendation_prompt.py. The LLM (Groq llama-3.3-70b-versatile) is provided with an explicit evidence payload containing: (1) ML risk score and confidence, (2) top SHAP risk drivers and protective factors, (3) NLP sentiment/burnout indicators, and (4) exact HR policy chunks retrieved from ChromaDB vector store. The system prompt contains explicit negative constraints instructing the model to ONLY use provided evidence, return `null` for absent policy references, and output structured JSON. We set `temperature=0.1` for deterministic outputs.

Q4: What happens during CSV employee import if a user uploads malformed data or malicious formulas?

Ideal Answer: Employee import uses a 2-stage governance pipeline in importGovernanceService.js. During the preview phase, the buffer is parsed using csv-parse/sync. We enforce a 5MB size limit, 5,000 row cap, and 50 column cap. Every cell value undergoes Formula Injection Protection via sanitizeCellValue, which automatically prefixes any string starting with =, +, -, @, \t, or \r with a single quote (') to prevent RCE in Excel. Schema validation checks required fields, email format regex, and numeric salary constraints. Rows with errors are flagged in a validation report, while valid rows are staged in an Import document. The user reviews errors before calling commit, which idempotently creates missing departments and upserts employee records bound strictly to the tenant's organizationId.

Q5: What are the primary system performance bottlenecks under high load, and how would you scale it?

Ideal Answer: According to our Autocannon load-testing benchmarks in docs/Load-Testing-Report.md, the primary bottleneck is unindexed Employee.find() queries on the 1,470-doc collection during dashboard requests, causing throughput to degrade from 18.6 to 8.9 req/s under 150 concurrent connections (p50 latency increasing to 7.8s). To scale the platform to 100,000+ employees and 1,000+ organizations, we would: (1) Add Redis caching for user auth sessions and employee dashboard aggregate queries, (2) Transition ML inference and LLM recommendation generation from synchronous API endpoints to asynchronous background task queues using Celery/RabbitMQ or Redis Streams, (3) Implement database read-replicas and compound index optimization, and (4) Deploy FastAPI AI workers auto-scaled on GPU/CPU instances.

9. Final Cheat Sheet & AI Knowledge Transfer Checklist

30-Second Interview Memory Recall (Top 9 Facts):

- **Core Architecture:** Express Backend (Port 5000) + FastAPI AI Service (Port 8000) + React Client (Port 80) + MongoDB 7 + ChromaDB Vector Store.
- **ML Algorithm:** Scikit-Learn Logistic Regression (C=0.01, penalty='l2', solver='lbfgs') with Isotonic Regression probability calibration.
- **Key Metrics:** 82.31% Accuracy, 35.44% Precision, **96.55% Recall**, 0.5185 F1-Score, 0.8884 ROC-AUC, 0.3213 PR-AUC at 0.07 decision threshold.
- **Dataset & Split:** 1,470 non-deleted employees (147 terminated / 10% attrition rate), 28 features (23 numerical + 5 categorical), 80/20 stratified train/test split.
- **Explainability (XAI):** Singleton ShapExplainerCache initialized on startup in worker thread; uses shap.LinearExplainer with 100 synthetic background samples.
- **GenAI Stack:** Groq API llama-3.3-70b-versatile (temp 0.1, max_retries=0) + ChromaDB RAG policy retrieval + 4-tool evidence prompt returning structured JSON.
- **Multi-Tenancy & Security:** Strict organizationId isolation resolved from verified DB User record in authenticate.js; CSV formula injection sanitization escaping =, +, -, @.
- **Scale & Test Suite:** 188 API endpoints (142 Express + 46 FastAPI), 9 RBAC roles, 34 Node test files (100% security/IDOR isolation pass rate).
- **Known Bottleneck:** Unindexed employee list queries under load (18.6 → 8.9 req/s, p50 7.8s); future architecture requires Redis caching and async task queues.

AI Context Transfer Knowledge Checklist:

An offline AI model consuming this document can reliably answer:

✔ What is Retention AI's business & technical objective?
✔ What exact ML model, calibration, features, and hyperparameters are used?
✔ How was the 96.55% recall metric achieved and defended against precision trade-offs?
✔ How does multi-tenant isolation work and how was header-spoofing vulnerability resolved?
✔ How does 2-stage CSV import handle formula injection and duplicate validation?
✔ How do SHAP explanations and Groq Llama-3.3-70B RAG recommendations execute?
✔ What are the 188 API endpoints, 9 RBAC roles, 46 database models, and load-testing bottlenecks?